增量记录 • 接续 2026-11-06 报告

从"规划会不会被拒"到"该派给哪个后端"

2026 年 11 月 6 日晚 – 11 月 7 日 • 新增 88 次执行运行

changkong34@qq.com

上一版报告停在第四批（80 次规划运行），结论是"需求里给一个可锚定的东西就够了"。 那之后做了三件事：把验收器补齐、把第二个后端接进来、 用 88 次执行运行回答"换模型值不值"。本文只记增量，不重复旧结论。

一 先修工具，再谈结论

往执行阶段推的第一天，连续三次失败，全部出在验收器，没有一次是模型的问题。

现象	真实原因	误导性
任务 failed，判据全挂	判据写作 `命令 `pytest …` 退出码 0`，识别逻辑只认裸命令开头，被当成"断言型"交给模型，而模型看不到命令输出	看起来像 worker 没做对
判据"输出受 verbose 控制"判 FAIL	只给 diff，`if verbose:` 是三行上下文之一，看不出块结构	看起来像代码有问题
`输出仅含 X` 必然 FAIL	只实现了"退出码 N"型，没有"命令+期望输出"型	看起来像 worker 多改了文件

> 这次最值钱的一条经验
>
> 验收器能力不足时，失败会伪装成"被验收者的错误"。 不逐条核对证据，这些都会被记进"模型成功率"，把系统缺陷算成模型差异。

对应的改动：命令判据兼容三种写法（裸命令 / 带 `命令 `…`` 前缀 / `运行: …`）； 新增"命令+期望输出"型判据（仅含 / 等于 / 含 三种模式）；支持 `python -c` 形态。

还有一个结构性问题值得单独记：最初"可判定"的判断只写在验收器里，契约层不认这种形态， 于是出现校验说不可判定（E014 拒稿）、执行时却按另一套逻辑。 现在两份定义合并到同一处——"可不可判定"和"怎么执行"必须共用一份规则。

测试从 57 涨到 122，代码 4,798 行。验收器的关键用例是"`cat a.txt b.txt` 必须判失败"， 否则"仅含"形同虚设。

二　接入第二个后端

原计划是三家厂商异构，但实测只有一家能被程序调用：联想小天（GUI）与 DeepSeek Harness（GUI） 都没有命令行入口，本地千问 1.7B 的权重用了私有量化格式，llama.cpp 读不了。

真正打通的是 同一个 Codex 外壳换后端。关键发现是 DeepSeek 原生支持 Responses API， 因此不需要协议转换、也不需要第三方切换工具：

```
-c model_provider=custom
-c model_providers.custom.base_url=https://api.deepseek.com
-c model_providers.custom.wire_api=responses
-c model_providers.custom.env_key=DEEPSEEK_KEY
-m deepseek-flash
```

踩过的坑：第一版抄了图形化切换工具生成的配置，里面是 `requires_openai_auth=true`， Codex 会去读自己的登录态（OpenAI 的 token），DeepSeek 不认，返回 401。 但报错里那句"your api key `****jHfg` is invalid"里的尾号，与我们真实的 key 尾号不一致—— 这个细节是唯一能区分"key 坏了"和"用错了 key"的线索。

另：原生模型 id 是 `deepseek-flash`，不是界面里显示的 `deepseek-v4.1-flash`， 后者会被 API 直接拒绝。

三　换后端，质量上没有可测差异

三条中等难度任务（改响应结构 / 跨文件改语义 / 改排序），每条都要求读懂现有代码、改两处、测试保持绿。 验收全部是确定性判据（`python -c` 断言行为 + `pytest` 回归），没有任何模型判分。 每任务重复 6 遍。

任务	OpenAI 中位	Pro 中位	Flash 中位
m1 响应改 `{"id": n}`	70s	84s	126s
m2 delete 布尔 + rm	91s	164s	120s
m3 列表降序	69s	97s	123s
总体（中位）	73s	122s	120s
相对 OpenAI	1.00×	1.68×	1.53×
通过率	18/18	17/18	18/18

质量：三者都做对了。Pro 那一次失败是被机器休眠打断的（墙钟 31,451 秒），不计入能力。

速度：换成 DeepSeek 一定更慢，而且显著。 Pro 与 OpenAI 配对 17 对，16 对更慢，符号检验 $p = 0.00014$； Flash 配对 18 对，14 对更慢，$p = 0.015$。换更轻的档位只把差距从 1.68 倍压到 1.53 倍。

一个必须写进局限的观察

两批的逐任务倍率互相反着：Pro 在 m2 最慢，Flash 在 m1 最慢，而 OpenAI 的基准两批都很稳。这说明"任务 × 模型"的交互在 n=6 下极不稳定。只能信总体倍率，不能写"某模型擅长某类任务"。

四　成本：Flash 是 Pro 的四分之一

官网定价页抓不到（302 跳转 / 429 限流），只能实测。方法是读余额、跑 8 次真实任务、等结算、再读余额。

后端	次数	通过	总花费	元 / 任务
Codex + Flash	8	8/8	¥0.31	¥0.039
Codex + Pro	8	8/8	¥1.24	¥0.155
差额	—	—	¥0.93	4.0 倍

余额 52.41 → 50.86 元，正好 ¥1.55，与两臂之和完全对上——账能对平，说明测法可信。

为什么放弃"先测单价再乘 token"

缓存价低到测不出来。116k token 的命中价约 0.02 元/百万，要凑出余额 0.01 元的变化得跑几千万 token。 而任务成本本身就在 0.04 – 0.16 元量级，直接测它精度反而更好（误差约 2%）。

三个会把结论带偏的坑

1. 余额是延迟批量入账的。调完立刻读是 `0.00`，实测滞后 1 – 2 分钟。 按"调用前后立即读数"测，会得出"免费"的荒谬结论。
2. 缓存污染冷启动测量。同一个 prompt 第二次起命中率 100%，测出来的是缓存价，不是标价。
3. 计费口径与耗时口径不能混批。成本那批的耗时（192s / 217s）明显高于对照批（120s / 122s），因为没挂验收器、仓库状态也不同。耗时只信第六批，成本只信第七批。

五　这两条线合起来说了什么

结论

DeepSeek 这条路选 Flash。质量与 Pro 同为满分，便宜 4 倍，还略快一点。 Pro 在这一档任务上没有任何理由存在。

但换 DeepSeek 本身是有代价的选择：它比默认后端慢 1.5 倍。 省的是钱，不是时间。这个交换在"额度受限"时成立，在"延迟敏感"时不成立。

更重要的是对 Q2 的否定回答：换更强的模型没有换来质量。 三条任务上两个后端都做对了，差异只在速度与成本。这意味着"按能力分派"在这类任务上没有可分的差异—— 而成本和延迟用配额规则就能算，不需要模型去判断。

这与第一部分的结论构成一种对称：决定成败的是"判据能不能被机器核对"，不是"模型够不够强"。需求写得含糊，任何后端都会反复返工；需求写得可锚定，便宜的后端也能一次做对。

六　这些证据有多硬

把两条线的结论按证据强度排一下，避免把"方向一致"当成"已经证明"。

结论	证据	强度
需求详细程度呈 U 形	6 任务 × 5 档全平衡，每档 6 条，无单点例外	强
失败机制只有 E014	80 次规划、47 次拒稿，零其他错误码	强
锚点最优点在 1 个	24 次，0 vs 1 的符号检验 p = 0.031	中
换后端不损质量	3 任务 × 6 遍，两臂均满分；但有天花板效应	中
换后端慢 1.5–1.7 倍	35 对配对，16/17 与 14/18 同向，p 分别为 0.00014 与 0.015	强
Flash 比 Pro 便宜 4 倍	各 8 次，余额差 ¥0.31 / ¥1.24，账能对平	中
某模型擅长某类任务	两批倍率互相反着，n = 6 不稳定	不成立

"中"的共同原因是样本量：锚点实验 n = 8/档，成本实验各 8 次。 方向可信，效应量的具体数值不可信——尤其不要引用"1.68 倍"这类精确到小数点的数字当结论。

七　下一步

1. 加难度。当前存在明显的天花板效应（两边全过），测不出能力上限。 需要"改动跨三个文件 + 有隐含约束"的任务。
2. 补单价。本次只测了任务成本，没测单价；也没测 OpenAI 那条路的边际成本（订阅制，配额内为 0）。
3. 把"判据可执行性"做成检查项。那两类伪装成模型错误的失败目前没有规则覆盖，而它们都是可检测的。

数据：`experiments/results_medium6.csv`（Pro 36 次）、`results_flash6.csv`（Flash 36 次）、`experiments/cost/`（成本 16 次）。结论原文见 `experiments/FINDINGS.md` 第五至第七批。